

Seq2Seq(sequence-to-sequence)

정의

Encoder

Decoder

교사 강요(teacher forcing)

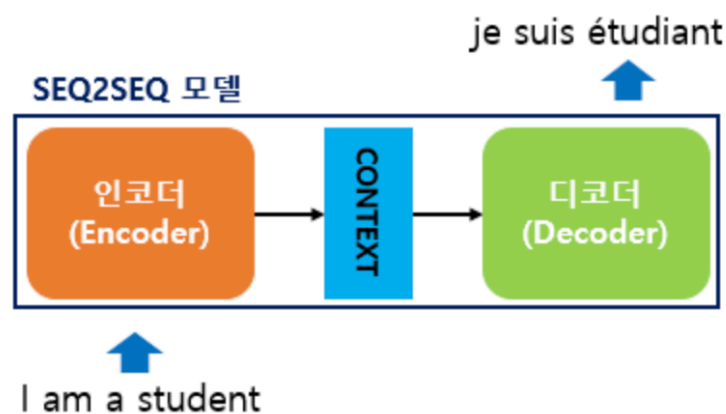
Decoder의 학습과 테스트 정리

정리

한계

정의

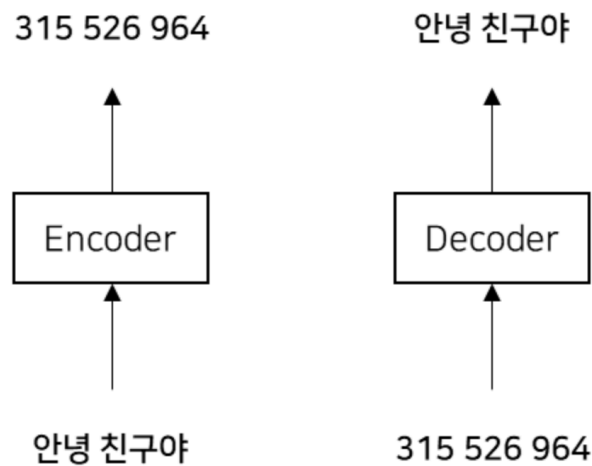
- 한 시퀀스를 다른 시퀀스로 변환하는 작업을 수행하는 딥러닝 모델 주로 자연어 처리(NLP) 분야에서 활용
 - 기계 번역, 문장 생성, 질의응답, 메일 자동 응답 등
- 인코더(Encoder)와 디코더(Decoder)라는 모듈로 구성되어 Encoder-Decoder 모델이라고도 함
- 두 모듈이 서로 협력하여 입력 시퀀스를 원하는 출력 시퀀스로 변환함



[그림-1] Seq2Seq 구성

- 인코더는 입력 데이터를 인코딩하고, 디코딩은 인코딩된 데이터를 디코딩함

예를 들면, 비밀편지를 쓸 때, 원래의 문장을 암호화해서 다른 사람이 쉽게 알아볼 수 없도록 바꾸는 과정이 필요합니다. 이 과정이 인코딩이 됩니다. "안녕 친구야"라는 문장을 "315, 526, 964"와 같은 숫자로 바꾸어 적는 것입니다. 디코딩은 인코딩과 반대로 다시 원래의 형태로 되돌려 주는 과정을 말합니다. "315, 526, 964"와 같은 문장을 다시 "안녕 친구야"라는 문장으로 바꾸어 읽는 것입니다.

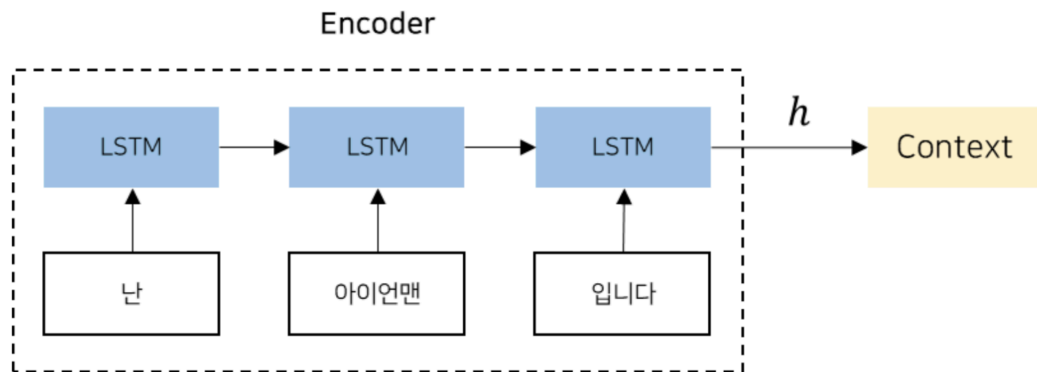


[그림-2]인코딩-디코딩 예시

Encoder

- 인코더는 일반적으로 RNN(Recurrent Neural Network)이나 LSTM(Long Short-Term Memory), GRU(Gated Recurrent Unit) 등의 **순환 신경망 구조를 사용하여 입력 시퀀스를 고정 길이의 벡터로 변환하는 역할을 수행**
- 이러한 구조는 시퀀스 데이터를 처리하는데 적합하며, 순차적인 정보를 효과적으로 인코딩할 수 있음
- 인코더는 아래의 그림같이 입력 시퀀스의 각 단어를 순차적으로 처리하면서 **각 단계에서 hidden state를 업데이트하고, 최종적으로 전체 입력 시퀀스를 대표하는 고정 길이의 벡터를 생성**
- 인코더의 LSTM 계층은 오른쪽(시간 방향)으로도 은닉 상태를 출력하고 위쪽으로도 은닉 상태를 출력하지만 이 구성에서 더 위에는 다른 계층이 없으니 LSTM 계층의 위쪽 출력은 폐기됨
- 인코더에서는 마지막 문자를 처리한 후 LSTM 계층의 은닉 상태 층인 Context 벡터를 디코더로 전달함

예를 들어 "난 아이언맨 입니다"라는 문장을 인코딩하여 고정 길이 벡터 h 를 생성하고 인코딩된 정보는 디코더에 전달됨



[그림-3] 인코딩 결과 고정 길이 벡터 h 생성

- 인코더가 생성한 벡터 h 는 LSTM의 마지막 hidden state
- 여기서 중요한 점은 고정 길이 벡터라는 것임

CONTEXT	0.15
	0.21
	-0.11
	0.91

[그림-4-1] context vector 예시

- 인코더가 인코딩 한다는 것은 임의의 길이의 문장을 고정 길이 벡터로 변환하는 작업을 의미함

h

난 아이언맨 입니다. —————→ [0.1, -0.5, -0.7, 0.9, 0.8]

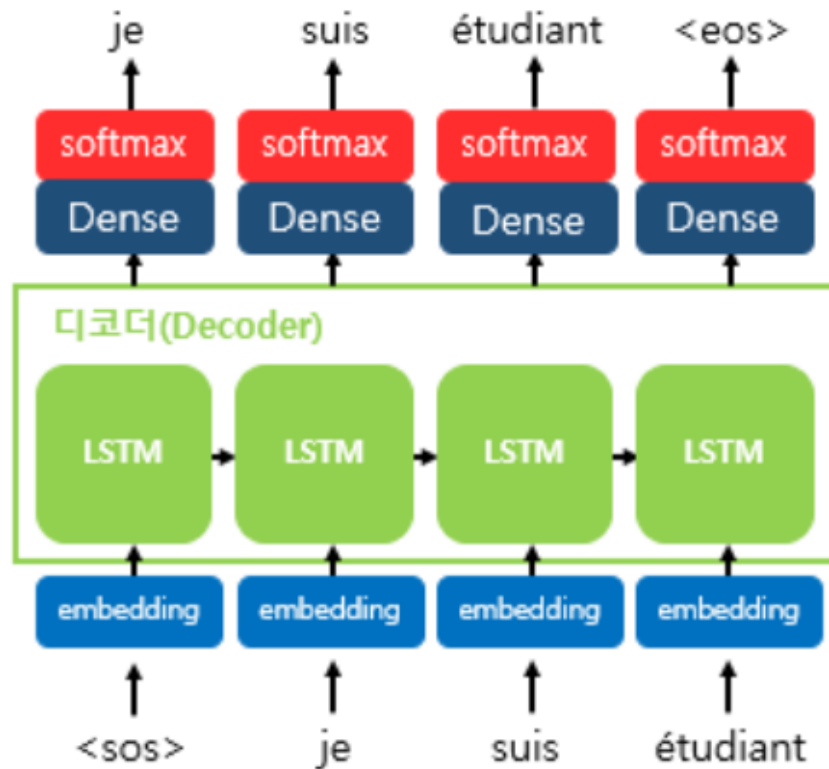
만나서 반갑습니다. —————→ [0.4, 0.2, -0.1, -0.5, 0.3]

우리는 공부를 합니다. —————→ [-0.9, 0.1, -0.2, -0.6, 0.8]

[그림-4-2] 고정 길이 벡터 h 예시

Decoder

- 디코더는 인코더의 출력인 고정 길이의 벡터를 기반으로 원하는 출력 시퀀스를 생성하는 역할을 수행
- 디코더 역시 RNN, LSTM, GRU 등의 순환 신경망 구조를 사용함
- 기존 순환 신경망 구조와 다른 점은 벡터 h 를 입력으로 받음
- 디코더의 예측은 일반적으로 소프트맥스 활성화 함수를 통해 확률 분포로 변환되며, 가장 확률이 높은 단어가 선택됨
- 이 과정을 반복하여 최종적으로 출력 시퀀스가 생성됨



[그림-5] 디코더 동작 원리

- 디코더는 입력 문장을 통해 출력 문장을 예측하는 언어 모델 형식임
 - 디코더 RNN 셀의 출력으로 다양한 단어에 대한 벡터 값이 나오며 그중 확률이 가장 높은 단어를 선택하기 위해 softmax를 통해 최종 예측 단어를 생성함
- <sos>는 문장의 시작(start of string)을 뜻하고 <eos>는 문장의 끝(end of string)을 뜻함
 - 구분 기호로 <go>, <start>, <s>, 등을 이용하기도 함
- 인코더로부터 전달받은 Context 벡터와 <sos>가 입력되면 그다음에 등장할 확률이 가장 높은 단어('je')를 예측함
- 다음 스텝에서는 이전 스텝의 예측 값인 'je'가 입력되고 'je' 다음에 등장할 확률이 가장 높은 단어('suis')를 예측함
- 이런 식으로 문장 내 모든 단어에 대해 반복함(이는 테스트 단계에서의 디코더 작동 원리임)
- Training 단계에서는 **교사 강요(teacher forcing)** 방식으로 디코더 모델을 훈련

교사 강요(teacher forcing)

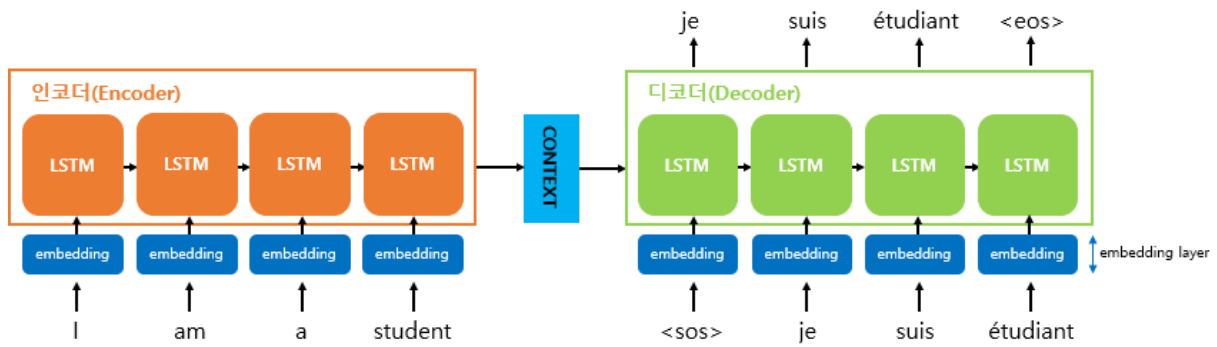
- 보통 RNN은 $(n-1)$ 스텝에서의 출력 값을 n 스텝의 입력값으로 사용함 즉, $(n-1)$ 스텝에서 RNN 모델이 예측한 값을 n 스텝의 입력값으로 사용함
- 교사 강요는 $(n-1)$ 스텝의 예측 값을 n 스텝의 입력값으로 사용하는 것이 아니라 $(n-1)$ 스텝의 실제값을 n 스텝의 입력값으로 넣어주는 방식
- 이유는 $(n-1)$ 스텝의 예측값이 실제값과 다를 수 있기 때문임
- 정확한 데이터로 훈련하기 위해 예측값을 다음 스텝으로 넘기는 것이 아니라 실제값을 매번 입력값으로 사용하며 이를 교사 강요라고 함
- 디코더의 훈련 단계에서는 교사 강요 방식으로 훈련하지만 테스트 단계에서는 일반적인 RNN 방식으로 예측함
- 테스트 단계에서는 Context 벡터를 입력값으로 받아 이미 학습된 디코더로 다음 단어를 예측하고, 그 단어를 다시 다음 스텝의 입력값으로 넣음.
- 이렇게 반복하여 최종 예측 문장을 생성함

Decoder의 학습과 테스트 정리

- 디코더의 학습 단계에서는 필요한 데이터가 Context 벡터와 <sos>, je, suis, étudiant
- 테스트 단계에서는 Context 벡터와 <sos>만 필요함
- 학습 단계에서는 교사 강요를 하기 위해 <sos>뿐만 아니라 je, suis, étudiant 모두가 필요하지만 테스트 단계에서는 Context 벡터와 <sos>만으로 첫 단어를 예측하고, 그 단어를 다음 스텝의 입력으로 넣음

정리

- 인코더와 디코더 역할을 하는 두 개의 LSTM으로 구성되어 있음
- 인코더는 입력 문장의 정보를 압축하는 기능을 하며 압축된 정보는 Context 벡터라는 형식으로 디코더에 전달됨
- 디코더는 학습 단계에서는 교사 방식(teaching force)으로 훈련되며, 테스트 단계에서는 인코더가 전달해준 Context 벡터와 <sos>를 입력값으로 하여 단어를 예측하는 것을 반복하며 문장을 생성함
- 역전파 과정에서는 컨텍스트 벡터를 통해 기울기가 디코더에서 인코더로 전달되어 모델이 학습됨



[그림-6] Seq2Seq 구조

- 딥러닝 모델은 문자보다 숫자에 대한 성능이 더 좋기 때문에 모든 문자는 숫자화해야 하며, 이를 워드 임베딩(Word Embeddings)이라고 함
- I, am, a, student라는 문자도 모두 숫자로 즉, 벡터로 표현해야 함
- 아래는 I, am, a, student에 대해 워드 임베딩을 한 결과인 벡터를 나타낸다. 아래는 각 단어에 대해 4차원으로 표현했지만 실제로는 몇백 차원으로 표현되어 있음

I	0.157
	-0.25
	0.478
	-0.78
am	0.78
	0.29
	-0.96
	0.52
a	0.75
	-0.81
	0.96
	0.12
student	0.88
	-0.17
	0.29
	0.48

[그림-7] 임베딩 결과 예시

한계

- 인코더가 입력 시퀀스를 하나의 **고정된 길이의 벡터로 압축**합니다. 이로 인해 입력 **시퀀스의 길이가 길어질수록 정보의 손실이 발생**
- RNN의 고질적인 문제인 **경사 소실 또는 폭발 문제가 존재**
- 이를 극복하기 위해 제안된 방법이 **어텐션 매커니즘**